

MATH 3190 Homework 9

Due April 9, 2026

Focus: Notes 10

Total Points: 70 + 10 (Self-Critique Comment and Uploading to GitHub) = 80

Note that your homework should be completed in Quarto. You can just add your answers and code to this document in the appropriate part. The file should be rendered as an html document or pdf document (html is quicker to render and you will need an installation of LaTeX to render as a pdf). You will “turn in” this homework by uploading the files, the .qmd and output file, to your GitHub `Math_3190_Assignment` repository in the `Homework/Homework_9` directory. Make sure all supporting files (if there are any) are in this directory as well. You should use `git` and `Unix` commands in a terminal for this (i.e. don’t manually upload files to GitHub). In addition, you will submit just your html or pdf file on Canvas to unlock the solutions.

Don’t forget to also put a self-critique comment on Canvas indicating what you did correctly and incorrectly and include any questions you may have about the assignment after submitting the assignment and thoroughly looking through the solutions.

Problem 1 (70 points)

Before we begin, please re-install the `math3190package` since there was an update to the `classifier_plot()` function that is needed on this assignment. Use either `devtools::install_github("rbrown53/math3190package")` or `remotes::install_github("rbrown53/math3190package")`.

The National Institute of Diabetes and Digestive and Kidney Diseases conducted a study on 768 adult female Pima Native Americans living near Phoenix. The purpose of the study was to investigate factors related to diabetes. The response variable in this study (test) was whether or not the patient showed signs of diabetes (1 = yes, 0 = no). One explanatory variable considered was the plasma glucose concentration (in mg/dL) at 2 hours in an oral glucose tolerance test (glucose). Many other potential explanatory variables were considered as well including body mass index (BMI). The goal here is to predict the diabetes status of the subject, which is in the `results` variable.

Part a (5 points)

Install and/or load the `faraway` package with the `pima` dataset. Set a seed and then split the data into a testing and a training set with the testing set containing 20% of the data values. When you do this, change the name of the variable “test” to “result” since it will be a bit confusing to call that test when we have a test dataset. Then make the `result` variable a factor so **R** treats it as categorical rather than numeric.

Part b (6 points)

- Fit a logistic regression model to the training set for predicting `result` from the `glucose` and `bmi` variables.
- Then use the `classifier_plot()` function in the `math3190package` (make sure you re-installed it!) to make a plot that shows the regions on the `glucose-bmi`-plane that are predicted to be 0 and 1. This shows how logistic regression divides the classes. Use the **training set** as the `data` input in this function.
- Next, obtain the predictions for the **test set** with a probability cutoff of 0.5 to predict if the person has diabetes. Create a confusion matrix using the `confusionMatrix()` function in the `caret` library. Make sure to put `positive = "1"` for this and all other confusion matrices on this assignment. Print all of the output from that function that shows the confusion matrix and many associated statistics.
- Write a sentence or two discussing how well this model performs in predicting on the test set.

Part c (3 points)

Now let's use naïve Bayes to get predictions. Use the `naive_bayes()` function in the `naivebayes` library to fit a model predicting `result` using `glucose` and `bmi`. Find the predicted probabilities of being a 0 and a 1 for the first observation in the training set.

Part d (9 points)

Now find the predicted probability of being a 0 and a 1 for the first observation in the training set “by hand” like we did in the notes on pages 187 and 188. To simplify this a bit, let's again only use the variables `glucose` and `bmi` here. Of course, use **R** to help you (the `dnorm()` function and some tidyverse functions will be especially nice), but don't use the `naive_bayes()` function here. Note: these variables we are using to predict are numeric, so we will assume a normal distribution for both of them (maybe not a good assumption, but let's go with it). After you get the probabilities, compare them to the ones you got in part (c). They should match.

Part e (3 points)

Now fit a naïve Bayes model predicting **result** from **glucose** and **bmi** using a kernel instead of assuming a normal distribution. You can do this by setting `usekernel = TRUE` in the `naive_bayes()` function.

Part f (5 points)

Now use the `classifier_plot()` function to make two plots: one using the first naïve Bayes from part (c) and one using the naïve Bayes with the kernel from part (e). Both should show the separation on the **training set**. Comment on how these models seem to perform in separating the results classes and if there seems like there may be any over-fitting here.

Part g (5 points)

Report the output of the `confusionMatrix()` function for both naïve Bayes models when predicting on the **test** set. Write a couple sentences comparing the confusion matrix statistics.

Part h (3 points)

Now let's use a support vector machine (SVM) for classifying diabetes using **glucose** and **bmi**. We'll use the `svm()` function in the `e1071` library. Fit a model with a **polynomial** kernel here with degree 3 (the default), a cost of 1 (also the default), and keep `scale = TRUE` (which is the default as well). Ideally, we would do a cross validation to select the cost, but we'll do that for the next kernel.

Part i (10 points)

Now let's use a different kernel, still predicting **result** from **glucose** and **bmi**. Let's use the radial kernel for the SVM. This has two tuning parameters: the cost and γ . Let's use the `train()` function in the `caret` package to do 5-fold cross validation (set a seed, please) to select both of those parameters. However, the `train()` function uses σ instead of γ and C instead of cost.

Use the `trainControl()` function with the `trControl` option and the `tuneGrid` option in the `train()` function to perform this cross-validation. For `sigma`, use `seq(0.1, 2, by = 0.1)` and for `C` also use `seq(0.1, 2, by = 0.1)`. Put `method = "svmRadial"` in the `train()` function as well so it knows to use a SVM with the radial kernel. You will need to install the `kernlab` package for this to work. Set a seed here as well.

By default, the “best” sigma and C values will be chosen with accuracy instead of Cohen’s kappa. Make sure to choose the best values using kappa (although both metrics may choose the same “best” values).

Part j (3 points)

Fit the SVM with the **radial** kernel using the `svm()` function and the cross-validated cost and γ found in part (i).

Part k (5 points)

Create the classifier plots for both SVM models and plot the training set and comment on how well these seem to separate the classes.

Part l (3 points)

Report the confusion matrix statistics for both SVM models when predicting the **test** set.

Part m (10 points)

Compare the output of the `confusionMatrix()` function for all of our classifiers: the logistic regression model, the naïve Bayes with a normal assumption, the naïve Bayes using kernel density estimation, the SVM with a polynomial kernel, and the SVM using a radial kernel. Which was best? Which was worst? How different were the results? You can focus on the Cohen’s κ and balanced accuracy for each one.